Title of the project 2: hangman game

Name of the student: Naga Rakshitha p v

Date of submission:4-02-2025

Title of the contents:

1.Introduction:

Hangman Game

Hangman is a word-guessing game where players guess letters to complete a hidden word within a limited number of attempts.

Significance:

- **Improves Logical Thinking**: Players use reasoning to guess words.
- **Enhances Vocabulary**: Helps learn new words and spelling.
- **Fun and Interactive**: Engages users in an interesting way.
- **Demonstrates String Handling in Python**: Involves working with strings, lists, and loops.

Objectives:

- Select a random word for the player to guess.
- Allow the player to guess letters one by one.
- Track correct and incorrect guesses.
- Limit the number of incorrect guesses before losing.
- Display the current progress of the word (e.g., `_ a _ a _`).

1.1Purpose of the Hangman Game

The **Hangman Game** is designed for both entertainment and educational purposes.

Key Purposes:

- **Enhances Vocabulary** – Helps players learn new words and spelling.
- **Improves Logical Thinking** – Encourages strategic guessing and problem-solving.
- **Teaches String Manipulation** – A great Python project for handling text and conditions.
- **Engages Users** – Provides a fun and interactive way to learn coding.
- **Develops Programming Skills** – Involves loops, conditions, and data structures like lists.

Functionalities:

- **Word Selection** – Chooses a random word from a predefined list.
- **User Guessing** – Allows the player to guess one letter at a time.
- **Tracking Progress** – Displays correctly guessed letters while hiding the rest (e.g., `_ a _ a _`).
- **Limited Attempts** – The player has a fixed number of incorrect guesses before losing.
- **Win/Loss Condition** – The game ends when the player either guesses the word or runs out of attempts.

Features:

- **Random Word Selection:** Uses Python's `random.choice()` to pick a word.
- **Text-Based Interface:** Displays progress and feedback in the console.
- **Lives System:** Limits the number of incorrect guesses (e.g., 6 attempts).
- **Visual Representation (Optional):** Can include ASCII art for the hangman drawing.
- **Enhancements:** Can be expanded to use a larger word database or a GUI version.

2.project overview

How the Hangman Game Works

The **Hangman Game** is a word-guessing game where the player tries to guess a hidden word letter by letter before running out of chances.

2.1.Step-by-Step Working of the Game

- **Select a Word**
    1. The program randomly selects a word from a predefined list.
- **Display the Hidden Word**
    1. The word is shown as underscores (_ _ _ _ _) representing each letter.
- **User Guesses a Letter**
    1. The player enters a single letter.
    2. If the letter is in the word, it is revealed in its correct position.

3. If the letter is **not in the word**, the player loses a life.

- **Tracking Incorrect Guesses**
    1. The game limits incorrect guesses (usually **6 chances**).
    2. A visual hangman (stick figure) may be drawn step by step as mistakes increase.
- **Win or Lose Condition**
    1. **Win:** If the player correctly guesses the word before running out of chances.
    2. **Lose:** If the player makes too many incorrect guesses. The word is then revealed.

2.2.Features of the Hangman Game

✅ **Random Word Selection** – Picks a new word each time from a list.

✅ **Letter-by-Letter Guessing** – Users input one letter at a time.

✅ **Lives System** – Players have limited incorrect guesses (e.g., 6).

✅ **Word Progress Display** – Shows correctly guessed letters in place (e.g., `_ a _ a _`).

✅ **Win/Loss Message** – Notifies the user whether they won or lost.

✅ **ASCII Art (Optional)** – Can display a visual hangman figure.

✅ **Difficulty Levels (Optional)** – Easy (longer words), Hard (shorter words).

2.3.Rules of the Hangman Game

1. The player must guess one letter at a time.
2. Correct guesses reveal the letter(s) in the word.
3. Incorrect guesses reduce the number of remaining attempts.
4. The game ends when the player:
    - **Guesses the word correctly (Wins)**
    - **Runs out of chances (Loses)**
5. The word is revealed at the end if the player loses.

2.4.User Interaction in the Game

1. Game Start

- The program welcomes the user and explains the rules.
- A random word is chosen, and its length is displayed as underscores.

2. Guessing Phase

- The user enters a letter.
- The program checks if the letter is in the word:
    - If **correct**, the letter appears in its correct place.
    - If **incorrect**, the player loses a life.

3. Game Feedback

- After each guess, the program updates the word display.
- The player is informed of remaining chances.

4. Game End

- The game ends when the word is guessed correctly or the player runs out of lives.
- A message displays whether the player **won or lost**.
- Optionally, the game can offer a **"Play Again"** feature.

3.Implementation Steps:

- **Import Required Modules**
    1. Use `random` to select a word from a predefined list.
- **Define the Word List**
    1. Create a list of words for the game.
- **Initialize Game Variables**
    1. Keep track of the guessed letters, attempts left, and correct guesses.
- **Game Loop**
    1. Ask the player to input a letter.

2. Check if the letter is in the word.
3. Update the displayed word and attempts accordingly.

- **Game End Conditions**
  1. If the player correctly guesses all letters → **Win**.
  2. If the player runs out of attempts → **Lose**.

3.1.Possible Enhancements

For Hangman Game:

✅ **Add a Hangman Drawing** – Use ASCII art to visualize the progress.

✅ **Increase Word Database** – Load words from an external file.

✅ **Add Difficulty Levels** – Easy (longer words), Hard (shorter words).

3.2.Technical Details of Implementation

Modules Used:

- `random` – To select a random word from a predefined list.

Technical Workflow:

- **Random Word Selection:**
  1. Uses `random.choice()` to pick a word from a predefined list.
- **Hidden Word Representation:**
  1. Converts the word into a list of underscores (_ _ _ _ _).
- **Game Loop:**
  1. Repeats until either the word is guessed or attempts run out.
  2. Accepts a letter from the player.
  3. Checks if the letter exists in the word.
  4. Updates the displayed word if correct.
- **Tracking Guesses and Attempts:**
  1. Stores guessed letters in a set to prevent duplicate inputs.
  2. Decreases attempts if the letter is incorrect.
- **Win/Loss Handling:**
  1. If all letters are guessed → **Player Wins**.

2.If attempts reach zero → **Player Loses**, and the correct word is revealed.

3.3.Code snippets:

```
import random

words = ["python", "hangman", "developer", "computer", "programming"]
word = random.choice(words)
hidden_word = ["_"] * len(word)
attempts = 6
guessed_letters = set()

print("Welcome to Hangman!")
print(" ".join(hidden_word))

while attempts > 0 and "_" in hidden_word:
    guess = input("\nGuess a letter: ").lower()

    if len(guess) != 1 or not guess.isalpha():
        print("Please enter a single letter.")
        continue

    if guess in guessed_letters:
        print("You've already guessed this letter!")
        continue

    guessed_letters.add(guess)

    if guess in word:
        for i in range(len(word)):
            if word[i] == guess:
                hidden_word[i] = guess
        print("Correct guess!")
```

```
    else:
        attempts -= 1
        print(f"Incorrect! {attempts} attempts left.")

    print(" ".join(hidden_word))

if "_" not in hidden_word:
    print("Congratulations! You guessed the word:", word)
else:
    print("Game over! The word was:", word)
```

Algorithms and Methods Used in the Code:

the **algorithm** for Hangman follows an interactive loop-based approach:

3.4.Algorithm:

1. **Word Selection:**
    - Pick a random word from a predefined list using `random.choice()`.
2. **Initialize Game Variables:**
    - Represent the word as underscores (_ _ _).
    - Set `attempts = 6` (or another limit).
    - Create an empty set to store guessed letters.
3. **Game Loop (While Attempts > 0 & Word Not Guessed):**
    - Accept a **single letter** input from the user.
    - Validate input (check if it is a letter and not guessed before).
    - If the letter is in the word:
        - Replace corresponding underscores with the letter.
    - If the letter is incorrect:
        - Reduce the number of attempts.
        - Optionally display a hangman drawing.
    - Display updated word progress.
4. **End Condition:**
    - If all letters are guessed, the player **wins**.

- If attempts reach **zero**, the player **loses**, and the word is revealed.

4.Code structure

4.1.Methods Used:

- `random.choice(list)`: Picks a random word from a list.
- `set()`: Stores guessed letters to prevent duplicate inputs.
- `str.join(list)`: Converts a list of characters into a string for display.

- **Main Module (hangman.py)**: Contains the core game logic.
  - **Functions:**
    - `play_game()`: Main function that starts the game and controls the flow (taking inputs, showing the word status, and managing guesses).
    - `display_word(word, guessed_letters)`: Displays the current state of the word, with unguessed letters replaced by underscores.
    - `check_guess(guess, word, guessed_letters)`: Validates a guessed letter and checks if it's part of the word.
    - `is_game_over()`: Checks if the game has ended, either through winning or losing.
  - **Helper Functions**:
    - `get_random_word()`: Picks a random word for the game from a predefined list or external file.
    - `validate_input(input)`: Ensures that the player inputs a single letter and no other characters.
- **Game Flow**:
  - Players take turns guessing letters.
  - A limited number of incorrect guesses (usually 6-7) before losing.
  - Displays the current state of the word with underscores and filled-in letters after each guess.
- **Libraries Used**:
  - `random` for choosing a word from a predefined list.
- **Directory Structure:**
- `bash`
- `CopyEdit`

```
hangman_game/
│
├── hangman.py                    # Core logic for Hangman game
├── README.md                     # Project documentation and usage
instructions
├── requirements.txt              # List of dependencies (if any)
├── words.txt                     # A text file containing a list of
words for the game
└── tests/
    └── test_hangman.py           # Unit tests for game logic (e.g.,
checking if game ends correctly)
```

4.2.How the Components Interact:

- **`hangman.py` (Core Game Logic)**:
  - **Role**: This file contains the main game loop, which includes all the logic for playing Hangman.
  - **Interactions**:
    - The `play_game()` function is the entry point for starting a new game. It orchestrates the game flow by calling functions to display the word, accept guesses, and check for game over conditions.
    - `display_word()` updates the game display to show the word with guessed letters and underscores for unguessed ones.
    - `check_guess()` validates the player's guess (ensures it's a single letter, checks if it's in the word, and updates the status).
    - `get_random_word()` selects a word for the game from the `words.txt` file, where each word is listed on a new line.
    - `is_game_over()` checks if the user has won or lost by evaluating the number of incorrect guesses or if all letters in the word have been guessed.
- **`README.md` (Documentation)**:
  - **Role**: Provides instructions on how to play the Hangman game, along with how to run the game on a local machine.
  - **Interactions**:
    - Like in the password generator, this file is used to explain the purpose of the project, setup steps, and dependencies.
- **`requirements.txt` (Dependencies)**:

- **Role**: If any external dependencies are used (e.g., additional libraries for user interface or enhanced functionality), they would be listed here.
- **Interactions**:
  - Developers can install the required libraries by running `pip install -r requirements.txt`.

- **`words.txt` (Word List)**:
  - **Role**: Contains a list of words that will be randomly selected for each game session.
  - **Interactions**:
    - `get_random_word()` reads the file and picks a random word for the game each time a new game is started.
- **`tests/test_hangman.py` (Testing)**:
  - **Role**: Contains unit tests to verify the functionality of the Hangman game.
  - **Interactions**:
    - Tests can be written for functions like `check_guess()`, `is_game_over()`, and `get_random_word()`. These tests ensure the game behaves correctly under different scenarios, such as when a player guesses the same letter repeatedly or when they win the game.

5.Testing

Testing Methodology for Hangman Game

For the **Hangman Game**, the focus is on ensuring that the **game logic**, **user input validation**, and the **word selection process** are functioning properly.

5.1. Types of Testing Used:

- **Unit Testing**:
  1. Unit tests in this context are used to test small parts of the Hangman game. This includes functions like:
     1. `display_word(word, guessed_letters)` to ensure it correctly displays the word with underscores for unguessed letters.
     2. `check_guess(guess, word, guessed_letters)` to test if the guess is correctly handled (whether it's a correct guess or not).

3. `is_game_over()` to check if the game correctly determines whether the player has won or lost.

- **Boundary Testing**:
    1. Test cases for when a player guesses invalid input (non-alphabetic characters, empty strings, etc.) to make sure the system handles such input gracefully.
    2. Testing edge cases like winning the game after one correct guess or losing after a certain number of wrong guesses.
- **Integration Testing**:
    1. Ensure the interaction between game logic functions, such as displaying the word and accepting guesses, works seamlessly.
    2. Test if the combination of word selection, user input, and game end condition functions correctly.
- **Functional Testing**:
    1. Ensure the game works as expected in real scenarios: the word is displayed with underscores, guesses are accepted, and the game ends correctly either with a win or loss.
- **Usability Testing**:
    1. While more subjective, you can ask users to play the game and give feedback on ease of use, instructions, and user interface (if any).

5.2.TEST cases

The Hangman game involves guessing letters to figure out a word. Here's how you can test different aspects of the game:

- **Boundary Test Cases**

- **Empty word**: Test the case where the word to guess is empty (though this is generally not valid for a Hangman game).
    - Input: `play_game("")` (ensure the game handles it properly).
- **Single-letter word**: Test with a single letter word.
    - Input: `play_game("A")`
- **Word with all unique letters**: Test a word with unique characters and no repeats.
    - Input: `play_game("WORD")`

- **Word with repeated letters**: Test a word with repeated letters to ensure they appear correctly.
    - Input: `play_game("BOOK")`
    - **Positive Test Cases**
- **Correct guess**: Test a scenario where the user guesses a letter correctly.
    - Input: `guess_letter("E")` (when the word is "ELEPHANT" and "E" is correct).
    - Expected: Game state should reflect that "E" has been guessed correctly, updating the word.
- **Full word guess**: Test when the user guesses the full word correctly.
    - Input: `guess_full_word("APPLE")` (if the word is "APPLE").
- **Game victory**: Ensure the game recognizes when the player has won (correct word guessed).
    - Input: After several correct guesses (e.g., `guess_letter("A")`, `guess_letter("P")`), the word "APPLE" should be revealed.
    - **Negative Test Cases**
- **Incorrect guess**: Test when the user guesses a letter incorrectly.
    - Input: `guess_letter("Z")` (when the word is "BANANA" and "Z" is not in the word).
    - Expected: The number of attempts should decrease, and the incorrect guess should be logged.
- **Already guessed letter**: Ensure the game doesn't allow guessing the same letter more than once.
    - Input: `guess_letter("A")` (if "A" has already been guessed).
    - Expected: The game should either ignore the input or notify the user that the letter has been guessed already.
    - **Edge Cases**
- **Guessing non-alphabetic characters**: Test the game with non-alphabetic characters to ensure it handles them properly.
    - Input: `guess_letter("1")` or `guess_letter("#")`
    - Expected: The game should either reject it or prompt for valid input.
- **Guessing when no attempts are left**: Test the behavior when the user runs out of attempts.

- Input: After making all incorrect guesses, input an incorrect letter.
- Expected: The game should end, and the user should be notified of a loss.
- **Performance Test Cases**

- **Stress test with long words**: Test the game with a very long word to check if it can handle long input.
  - Input: `play_game("SUPERCALIFRAGILISTICEXPIALIDOCIOUS")`
- **Handling many incorrect guesses**: Test when a user makes several incorrect guesses.
  - Input: `guess_letter("X")` multiple times until attempts are exhausted.

5.3.Testing Phase Results:

- **Boundary Testing:**
  - The game correctly handled single-letter words and words with repeated letters. It displayed the hidden word properly with underscores for unguessed letters.
  - It did not break with non-alphabetical characters and handled empty or invalid inputs gracefully.
- **Positive Test Cases:**
  - Correct guesses were reflected in the game state, and the word was updated as expected.
  - The game tracked the player's incorrect guesses and adjusted the number of attempts accordingly.
  - When the player guessed the full word correctly, the game ended with a victory message.
- **Negative Test Cases:**
  - The game appropriately rejected repeated or incorrect guesses, with clear feedback provided to the player.
  - Invalid inputs, such as numbers or special characters, were either ignored or flagged with an error message.
  - The game correctly handled the case where the player ran out of attempts, and a "game over" message was displayed with the correct word revealed.
- **Edge Cases:**

- The game worked well with very short (single-letter) and long words, even with complex and challenging words like "SUPERCALIFRAGILISTICEXPIALIDOCIOUS."
- The game provided an appropriate user experience even after multiple incorrect guesses, ensuring users understood the game's state.

- **Performance Testing:**
  - The game performed well with longer words and did not suffer from performance degradation or bugs.
  - It handled many incorrect guesses efficiently without slowing down or crashing.

6.Results and discussion:

6.1.How the Project Performed:

- **Strengths:**
  - The game successfully passed all functional tests and handled edge cases without issues.
  - It provided a smooth, engaging user experience with clear feedback for correct and incorrect guesses.
- **Weaknesses:**
  - Minor improvements can be made to ensure that error messages for invalid inputs (such as numbers or special characters) are more user-friendly.
  - It could benefit from additional features, such as a graphical user interface (GUI) or the ability to customize word lists, which would enhance its usability and engagement.

6.2.Key Features Achieved:

- **Game Mechanics:**
  - The game implemented core Hangman mechanics: the player guesses letters and the game shows blanks for unguessed letters and a graphical representation of attempts remaining.
- **Word Selection:**

- The game randomly selects a word and provides users with a chance to guess one letter at a time. It allows users to win if they guess the word correctly or lose when their attempts are exhausted.

- **Input Validation:**
  - The game handled inputs gracefully, ensuring that non-alphabet characters were rejected, and players could not guess the same letter twice.
- **User Feedback:**
  - The game provided clear feedback about the player's progress, with updates after each guess and game-over or win notifications.
- **Error Handling:**
  - Invalid guesses or repeated guesses were flagged, providing users with useful messages.

6.3.Outcomes Achieved:

- **Fun and Interactive Experience:** The game delivered a fully functional interactive game with the classic Hangman mechanics, providing both entertainment and an engaging user experience.
- **Functionality:** All test cases, including edge cases (e.g., single-letter words, invalid inputs), were successfully passed.
- **User-Centric Design:** The user could easily interact with the game and receive meaningful feedback on the progress of their guesses.
- **Performance:** The game performed well even with longer words, making it scalable for more challenging gameplay.

6.4.efficiency:

- **Algorithm Efficiency:**
  - The Hangman game operates efficiently with a time complexity of O(n) for most of the core operations (e.g., checking if a letter exists in the word, updating the guessed word). These operations are simple and do not require complex algorithms or data structures.
  - The efficiency of the game mainly depends on how the word is selected, how the player's guesses are handled, and how the game state is updated. Each of these operations is efficient with respect to game complexity.

- **Memory Efficiency:**
    - The game maintains a small set of variables to track the state of the game, including the word to be guessed, the correctly guessed letters, and the number of attempts remaining.
    - The memory usage is minimal since there's no need to store large datasets, and the game's data (like word and attempts) are stored in simple variables or arrays.

6.5.Performance:

- **Speed:**
    - The Hangman game is responsive, even when handling long words or multiple incorrect guesses. There is no noticeable lag, and the game updates in real-time as the user makes guesses.
    - Given that the game involves basic input/output operations (input for guesses, output for feedback), performance is not a major concern. It will work equally well for small and large words, with a typical game duration depending on the word length and the player's guesses.
- **Scalability:**
    - The game can handle various word lengths and complexities without any performance degradation. The gameplay experience remains smooth, even with longer words like "SUPERCALIFRAGILISTICEXPIALIDOCIOUS."
    - While scaling this game for multiplayer or adding complex graphics would require more resources, the current version of the game scales well for single-player text-based gameplay.

6.6.User Experience:

- **Engagement:**
    - The Hangman game provides a fun and interactive experience by allowing players to guess letters, keep track of their remaining attempts, and reveal the word gradually. The suspense of the game is a key component of user engagement.
- **Feedback:**

- Clear and timely feedback after each guess is essential for user engagement. The game should display whether the guess is correct or incorrect, update the word's revealed letters, and notify the player about the remaining attempts.
- When the game ends (either with a win or loss), the user is provided with the appropriate message (e.g., "You Win!" or "Game Over"), which is crucial for a satisfying user experience.

- **Error Handling:**
  - Input validation is essential for preventing invalid guesses. The game should handle cases where the user enters non-alphabetical characters or repeats guesses. If the player enters an invalid guess, an error message or prompt should notify them.
- **User Interface:**
  - If the game is text-based (CLI), the user experience is simple and direct, but might feel basic. If it has a graphical interface (GUI), it could provide more interactive elements, such as images of the hangman figure being drawn with each incorrect guess.
  - The layout should be intuitive, with clear instructions for how to play the game and easy-to-read output displaying the current game state.

7.Conclusion:

7.1.Summery of key points:

- **Efficiency:**
  - Simple algorithms (O(n) time complexity) for handling guesses, checking word progress, and tracking attempts.
  - Low memory usage, as it only stores essential game state data (word, guesses, attempts).
- **Performance:**
  - Smooth and responsive gameplay, with no lag or delays, even with long words.
  - The game performs well with various word complexities and multiple incorrect guesses.
- **User Experience:**

- Provides an engaging experience with clear feedback after each guess.
- Handles invalid inputs effectively (e.g., non-alphabetical characters), ensuring a smooth user experience.
- Text-based interface is simple, but could be enhanced with a graphical UI for a richer experience.

7.2.General Conclusion:

- **Efficiency:** Both projects are optimized for their respective tasks with minimal memory consumption and quick execution.
- **Performance:** Both projects are fast and scalable, meeting expectations for their use cases (password generation and word guessing).
- **User Experience:** Both projects are user-friendly, offering clear feedback, customization options, and error handling to ensure a smooth experience.

7.3.Reflection on the Success of the Project in Meeting the Objectives:

Objectives:

- **Create an interactive game** that allows users to guess letters in a word and track their progress.
- **Provide clear feedback** on correct and incorrect guesses.
- **Handle invalid inputs** and repeated guesses without crashing or confusion.

7.4.Success in Meeting Objectives:

- The **Hangman Game** successfully met its core objectives:
  - **Interactive Gameplay:** The game allowed users to guess letters, revealed correct guesses in the word, and displayed incorrect guesses with remaining attempts.
  - **Feedback:** The game provided immediate feedback after each guess, showing updated word progress and remaining attempts, making the game engaging.
  - **Input Validation:** Invalid guesses (non-alphabetic characters or repeated guesses) were handled gracefully, with clear error messages to guide users.

7.5.Reflection:

- The project was successful in creating an enjoyable and interactive Hangman game that fulfilled the basic requirements. The game provided a smooth, engaging user experience by keeping track of guesses and offering feedback throughout the game.
- One area for enhancement would be the addition of a graphical interface (GUI) to make the game visually more appealing and intuitive, and adding difficulty levels or an option for different word categories could increase replayability.

7.6.Future Improvements and Enhancements:

User Interface Enhancements

- **Graphical User Interface (GUI):**
  - Implement a GUI version using Tkinter, PyGame, or a web-based UI for an enhanced visual experience.
- **Animated Hangman Drawing:**
  - Introduce an animated hangman drawing that updates dynamically as incorrect guesses are made.

. Game Mechanics Enhancements

- **Difficulty Levels:**
  - Implement difficulty settings (Easy, Medium, Hard) by varying word length and allowed attempts.
- **Themed Word Categories:**
  - Allow players to choose word categories (e.g., Animals, Science, Technology) for a more customized experience.
- **Multiplayer Mode:**
  - Add a multiplayer option where one player inputs a word, and the other guesses it.

. Performance and Features

- **Leaderboard and Scoring System:**
  - Introduce a points-based system and track high scores for competitive play.

- **Sound Effects and Background Music:**
  - Add sound effects for correct/incorrect guesses and background music to improve engagement.
- **Hint System:**
  - Provide optional hints to help players if they are stuck.

8.References and Resources Used:

8.1.Websites & Online Documentation

- **Python Official Documentation** – Used for understanding Python's built-in libraries.
  - https://docs.python.org/3/
- **W3Schools (Python Section)** – Reference for string manipulation, loops, and random module usage.
  - https://www.w3schools.com/python/
- **GeeksforGeeks** – Articles and tutorials on implementing Hangman and password generators.
  - https://www.geeksforgeeks.org/
- **Real Python** – Guides and best practices for Python programming.
  - https://realpython.com/
- **Stack Overflow** – Helped in debugging issues and finding alternative solutions.
  - https://stackoverflow.com/

9.Appendices:

9.1.Code for Hangman Game

```
import random

# List of words
words = ["python", "developer", "hangman", "computer", "algorithm"]

def choose_word():
    return random.choice(words)
```

```
def display_word(word, guessed_letters):
    return "".join([letter if letter in guessed_letters else "_" for letter in word])

def hangman():
    word = choose_word()
    guessed_letters = set()
    attempts = 6

    print("Welcome to Hangman!")

    while attempts > 0:
        print("\nWord:", display_word(word, guessed_letters))
        guess = input("Guess a letter: ").lower()

        if guess in guessed_letters:
            print("You already guessed that letter!")
        elif guess in word:
            guessed_letters.add(guess)
            print("Correct guess!")
        else:
            attempts -= 1
            print(f"Wrong guess! Attempts left: {attempts}")

        if set(word) <= guessed_letters:
            print("\nCongratulations! You guessed the word:", word)
            return

    print("\nGame Over! The word was:", word)
#run the game
hangman()
```